§ 9. Персистентные деревья поиска и кучи

Дерево отрезков имеет заранее заданную форму: высота ограничена независимо от значений. У дерева поиска форма зависит от порядка изменений. У кучи мы вообще не требуем порядка между левым и правым поддеревьями, зато можем ограничить длину пути слияния. Поэтому одна фраза «скопируем путь» ещё не даёт логарифмической оценки.

В этой главе построим неизменяемое множество на BST и персистентную очередь с приоритетом на левосторонней куче. Для каждого интерфейса отдельно докажем логическую корректность и стоимость. Затем сравним левостороннюю и биномиальную кучи и разберём особенность слияния двух версий, имеющих общие узлы.

9.1. Два интерфейса и два разных порядка

Множество поддерживает `contains(root,x)`, `insert(root,x)` и `erase(root,x)`. В этой главе ключи уникальны: повторная вставка не меняет множество, удаление отсутствующего ключа тоже ничего не меняет. Все изменения возвращают корень. Чтение старой версии разрешено после любого числа новых операций.

Очередь с приоритетом хранит мультимножество: одинаковые ключи допустимы и считаются разными элементами. Интерфейс содержит `top`, `push`, `pop` и `meld(a,b)`. Операция `pop` удаляет одно вхождение минимума. `meld` объединяет мультимножества с суммированием кратностей. Чтение минимума или удаление из пустой кучи — ошибка.

В BST все ключи левого поддерева меньше ключа корня, а правого — больше. Это позволяет выбрать одну ветвь поиска по сравнению. В min-куче ключ корня не превосходит ключи обоих поддеревьев. Из свойства кучи не следует, в какой ветви находится заданный ключ; быстрый поиск произвольного значения не входит в её контракт.

Редакторский пример. Из множества $\{2, 5, 8\}$ создадим одну версию вставкой 4, другую — удалением 5. Получатся $\{2, 4, 5, 8\}$ и $\{2, 8\}$. Для кучи со значениями $[2, 5, 8]$ те же действия нельзя описать через `erase(5)`: такого метода у нашего интерфейса нет. Здесь можно удалить только минимум 2.

9.2. Неизменяемые узлы BST

Используем `shared_ptr<const Node>`. Тип `const` запрещает изменение узла через опубликованную ссылку; счётчик ссылок управляет временем жизни. Это альтернативная инженерная модель по сравнению с индексным пулом § 8. Сохраняются только узлы, достижимые из живых корней и временных локальных ссылок.

Ключ и дети определяются в конструкторе. Функция `make` создаёт новый узел, а поиск читает существующие. Листинги ниже составляют полный класс из `code/09-persistent-trees.hpp`; нужны `<memory>`, `<utility>`, `<stdexcept>` и `<algorithm>`.

(c) Aleksey Tolstikov

```
class PersistentBST {
    struct Node;
public:
    using Root=std::shared_ptr<const Node>;
private:
    struct Node {
        int key; Root left,right;
        Node(int k,Root a,Root b):key(k),left(a),right(b) {}
    };
    static Root make(int k,Root a,Root b) {
        return std::make_shared<const Node>(k,a,b);
    }
public:
    static bool contains(Root r,int x) {
        while (r) {
            if (x==r->key) return true;
            r=x<r->key ? r->left : r->right;
        }
        return false;
    }
```

Один и тот же узел может иметь много владельцев в физическом графе памяти. Для конкретного корня его развёрнутое логическое представление всё равно является деревом поиска. Неизменяемость позволяет рассуждать о каждой версии отдельно, не копируя общие поддеревья.

Сохранение только корня достаточно, потому что всё состояние структуры достижимо из него. Если добавить внешний изменяемый словарь «ключ → родитель», эта гарантия исчезнет: словарь тоже понадобится версионировать. Указатели на родителей особенно неудобны, поскольку общий ребёнок может иметь разных родителей в разных версиях.

9.3. Вставка, удаление и сохранение старого порядка

Вставка повторяет обычный поиск места. По возвращении из рекурсии создаёт копии предков. Неизменённый ребёнок сохраняется. Если ключ уже найден, возвращается тот же корень, а проверка равенства ссылок распространяет отсутствие изменений вверх без новых узлов.

```
static Root insert(Root r,int x) {
        if (!r) return make(x,nullptr,nullptr);
        if (x==r->key) return r;
        if (x<r->key) {
            Root a=insert(r->left,x);
            if (a==r->left) return r;
            return make(r->key,a,r->right);
        }
        Root b=insert(r->right,x);
        if (b==r->right) return r;
        return make(r->key,r->left,b);
    }
```

Вставим 4 в дерево с корнем 5 и детьми 2 и 8. Получим новый лист 4 справа от 2, новый узел 2 и новый корень 5. Узел 8 останется общим. Вставка существующего ключа 2 вернёт исходный корень, не создавая новых узлов.

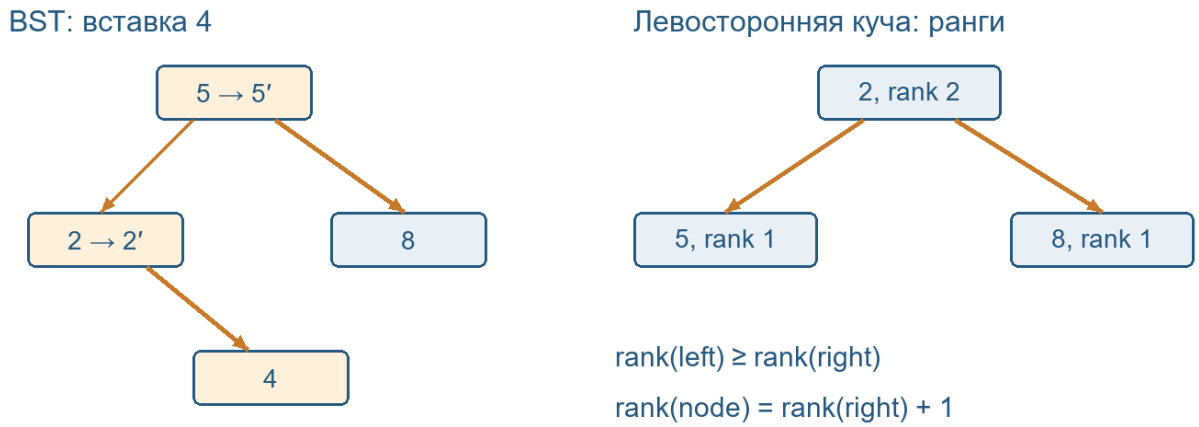


Рисунок 1: BST копирует поисковый путь. Левосторонняя куча контролирует ранг правого пути, а не общую высоту.

Для удаления есть три случая. У листа результат — пустая ссылка. Узел с одним ребёнком заменяется этим ребёнком. Если оба ребёнка существуют, возьмём минимальный ключ правого поддерева, создадим новый корень с этим ключом и удалим одно его вхождение справа. В множестве этот ключ встречается ровно один раз.

```
static Root erase(Root r,int x) {
        if (!r) return r;
        if (x<r->key) {
            Root a=erase(r->left,x);
            if (a==r->left) return r;
            return make(r->key,a,r->right);
        }
        if (x>r->key) {
            Root b=erase(r->right,x);
            if (b==r->right) return r;
            return make(r->key,r->left,b);
        }
        if (!r->left) return r->right;
        if (!r->right) return r->left;
        Root s=r->right;
        while (s->left) s=s->left;
        return make(s->key,r->left,erase(r->right,s->key));
    }
};
```

Почему выбор преемника сохраняет порядок? Все ключи слева меньше прежнего корня, а значит меньше преемника. Преемник — минимум справа, поэтому оставшиеся ключи справа строго больше него. Удаление рекурсивно сохраняет порядок внутри правого поддерева. Старый корень и его прежний преемник не изменяются.

Для нашего примера удаление 5 из исходного дерева создаёт корень 8 с левым ребёнком 2 и пустым правым ребёнком. Оно не затрагивает ветку, в которой ранее вставили 4. Удаление отсутствующего 6 пройдёт поиск и вернёт ту же ссылку, что получила операция.

Корректность доказывается по высоте: рекурсивное изменение строит правильное поддерево, новый родитель восстанавливает тот же порядок, остальные узлы неизменны. Для двух детей отдельно

используется свойство минимального ключа справа. Как и в § 8, доказательство включает сохранность всех старых корней.

9.4. Где появляется логарифм, а где его нет

Если высота исходной версии равна h, поиск занимает $O(h)$, изменение — $O(h)$ времени и новых узлов. Поиск преемника и его удаление проходят один путь дважды, что меняет константу, но не порядок оценки. Повторная вставка и удаление отсутствующего ключа новых узлов не создают.

В обычном BST может быть $h = n$. Последовательность вставок $1, 2, 3, \dots, n$ образует цепочку. Персистентное хранение всех промежуточных версий такого дерева требует $1 + 2 + \cdots + n = \Theta(n^2)$ узлов. Даже если каждый узел неизменяем, структура не становится сбалансированной.

Чтобы получить гарантированную логарифмическую высоту, можно использовать AVL-дерево. Вместе с ключом храним высоту; при возврате из рекурсии создаём новые узлы и выполняем функциональные повороты. При правом повороте вокруг y с левым ребёнком x сначала создаётся новый y с левым поддеревом $x.right$, затем новый x с этим y справа. Старые x и y не меняются.



Поддеревья A, B, C общие. Новые высоты вычисляются снизу вверх. При удалении балансировка может потребоваться на нескольких уровнях, поэтому недостаточно скопировать только два узла у первого поворота. При корректной AVL-балансировке путь и число новых узлов остаются $O(\log n)$.

Альтернатива — декартово дерево с независимыми случайными приоритетами. Приоритет хранится в узле и сохраняется при копировании; нельзя заново случайно назначать его каждому клону. Логарифмическая оценка здесь ожидаемая, а не гарантированная. Приведённый код намеренно показывает базовое BST: никаких скрытых гарантий AVL или treap у него нет.

Рекурсивная реализация может исчерпать стек на длинной цепочке. Для больших небалансированных данных нужен итеративный путь или балансировка. Персистентность не устраняет ограничения глубины рекурсии языка.

9.5. Левосторонняя куча: инвариант короткой правой ветви

У пустой кучи ранг 0. У непустого узла ранг равен единице плюс минимум рангов детей. Дополнительный левосторонний инвариант требует $rank(left) \geq rank(right)$. Поэтому $rank(v) = rank(v.right) + 1$: ранг равен числу узлов на правом пути до пустой ссылки.

У дерева ранга r не менее $2^r - 1$ логических узлов. Доказательство индукцией: при $r > 0$ оба ребёнка имеют ранг не меньше $r - 1$, значит содержат не менее $2^{r-1} - 1$ узлов каждый. Добавление корня даёт $2^r - 1$. Следовательно, правая ветвь имеет длину $O(\log(n + 1))$.

Заметьте, что всё дерево может быть высоким: длинная левая цепочка допустима. Для слияния нужна только правая ветвь, поэтому ограничивать полную высоту не требуется. Смешение ранга с высотой приводит к неправильной оценке и неправильной реализации `make`.

Представление снова неизменяемое. Конструктор `make` получает уже готовые поддеревья, при необходимости меняет их местами и вычисляет ранг. Перестановка не нарушает свойство min-кучи: оно симметрично относительно детей.

```
class PersistentLeftist {
    struct Node;
public:
    using Root=std::shared_ptr<const Node>;
private:
    struct Node {
        long long key; Root left,right; int rank;
        Node(long long x,Root a,Root b,int d)
            :key(x),left(a),right(b),rank(d) {}
    };
    static int rank(Root r) { return r ? r->rank : 0; }
    static Root make(long long x,Root a,Root b) {
        if (rank(a)<rank(b)) std::swap(a,b);
        int d=rank(b)+1;
        return std::make_shared<const Node>(x,a,b,d);
    }
public:
```

9.6. Слияние — единственная содержательная операция

Если одна куча пуста, возвращаем другую без выделения узлов. Иначе выбираем корень с меньшим ключом, пусть это a. Его ключ будет минимумом объединения. Левое поддерево a сохраняем, правое сливаем с b, затем восстанавливаем левосторонний инвариант через `make`.

```
static Root meld(Root a,Root b) {
        if (!a) return b;
        if (!b) return a;
        if (b->key<a->key) std::swap(a,b);
        return make(a->key,a->left,meld(a->right,b));
    }
```

Корректность состоит из трёх утверждений. Во-первых, мультимножество результата равно сумме мультимножеств аргументов: корень a и левое поддерево сохранены, рекурсивный результат содержит правое поддерево и всю кучу b. Во-вторых, $a.key$ не превосходит ключи всех этих элементов. В-третьих, `make` восстанавливает порядок рангов, не меняя состав элементов. Старые узлы только читаются.

Рассмотрим меру $rank(a) + rank(b)$. В каждом непустом рекурсивном шаге мы спускаемся по правой ссылке одного корня, а его ранг уменьшается на 1. Возможная перестановка аргументов сумму рангов не меняет. Значит, до пустой ссылки не более $rank(a) + rank(b)$ шагов. Слияние занимает $O(\log(n_a + 1) + \log(n_b + 1))$ времени и создаёт столько же узлов в худшем случае.

Никакого амортизационного переноса между версиями здесь нет. Каждая отдельная операция ограничена рангами собственных аргументов. Поэтому ветвление истории не ломает оценку, в отличие от повторного разворота очереди из § 7.

Вставка сливает исходную кучу с одноэлементной. Удаление минимума сливает детей корня. Доступ к минимуму просто читает ключ.

```
static Root push(Root r,long long x) {
        return meld(r,make(x,nullptr,nullptr));
    }
    static long long top(Root r) {
        if (!r) throw std::out_of_range("empty heap");
        return r->key;
    }
    static Root pop(Root r) {
        if (!r) throw std::out_of_range("empty heap");
        return meld(r->left,r->right);
    }
```

Класс продолжается диагностическим методом `valid` и закрывающей скобкой в полном заголовке.

Метод `valid` из полного заголовка используется только в проверках: рекурсивно проверяет порядок ключей и ранги. Это полный обход, его не нужно запускать внутри каждой рабочей операции. Иначе быстрые операции будут искусственно превращены в линейные.

9.7. Слияние родственных версий и кратности

Для обычных изменяемых куч слияние часто предполагает непересекающиеся владения памятью. Здесь старые корни сохраняются, поэтому нужно явно определить смысл общих узлов. Наш контракт — сумма мультимножеств. Если a и b содержат общий элемент по происхождению, он всё равно учитывается один раз от a и один раз от b.

Например, $h = [2, 5]$ и $g = push(h, 1) = [1, 2, 5]$. Результат `meld(h,g)` содержит $[1, 2, 2, 5, 5]$. Он не равен множеству $[1, 2, 5]$ и не является объединением уникальных идентификаторов задач. Для такого объединения нужен другой интерфейс и иной алгоритм устранения повторов.

Разрешён даже `meld(h,h)`: он удваивает каждую кратность. Физические узлы результата могут разделяться, но логическое дерево разворачивает каждую ссылку как отдельное вхождение. В этой модели размер n в оценке — количество логических элементов, а не число различных выделенных узлов. После многократного удвоения эти числа могут сильно различаться.

Цикл не возникает. Рекурсивный вызов получает меньший правый путь; новые узлы ссылаются на уже существующие или только что построенные поддеревья, ни один старый узел не получает обратной ссылки. Удаление минимума сохраняет корректные кратности и при общей памяти. Но полный обход для печати или `valid` может быть огромным относительно физического размера графа, поскольку посещает логические вхождения.

Для прикладной очереди задач часто правильнее запретить слияние родственных наборов по контракту. Тогда надо отслеживать происхождение элементов. Сам алгоритм кучи не умеет определять, должна ли одна и та же задача исполняться дважды.

9.8. Биномиальные кучи и осторожность с амортизацией

Биномиальное дерево B_0 состоит из одного узла. Дерево B_{k+1} получается связыванием двух B_k: корень с большим ключом становится ребёнком меньшего. У B_k ровно 2^k узлов, высота k, а ранги детей корня равны $k - 1, \dots, 0$.

Биномиальная куча хранит лес, в котором не более одного дерева каждого ранга. Наличие дерева ранга k соответствует единичному биту числа элементов. Слияние похоже на двоичное сложение: два дерева одного ранга связываются и дают перенос в следующий ранг. Если есть ещё и входящий перенос, одно дерево остаётся, а два других связываются.

При неизменяемом представлении связь создаёт новый корень победителя и новую ячейку списка детей; сами прежние поддеревья остаются общими. Список корней тоже должен быть неизменяемым. Тогда стандартное слияние лесов создаёт $O(\log(n_a + n_b + 1))$ новых ячеек и занимает столько же времени. Детали представления списков принципиальны: полное копирование массива всех потомков может ухудшить оценку.

Вставка одноэлементной кучи может вызвать цепь переносов. Если исходный размер $2^k - 1$, заняты все ранги от 0 до $k-1$, и одна вставка выполняет k связей. В линейной изменяемой истории такие переносы допускают амортизационный анализ. Но если m раз вставлять в одну и ту же старую версию размера $2^k - 1$, каждый раз придётся выполнить k связей: суммарно $\Theta(mk)$.

Поэтому для полной персистентности без специальной ленивой организации обещаем логарифмическую оценку отдельной вставки. Чтобы восстановить более сильные амортизированные гарантии, нужна отдельная конструкция с корректным разделением отложенной работы; обычного списка корней недостаточно.

9.9. Ресурсы и выбор реализации

Представление	Чтение	Изменение	Условие оценки
Обычное BST	поиск $O(h)$	$O(h)$ новых узлов	высота может быть n
AVL с копированием	поиск $O(\log n)$	$O(\log n)$	балансируется каждая версия
Левосторонняя куча	минимум $O(1)$	$O(\log n)$ для push/pop	ранг правой ветви
Биномиальная куча	минимум $O(\log n)$ без кеша	$O(\log n)$	корректные неизменяемые списки

Временные оценки наших методов считают обращения к узлам и выделение новых узлов. Управление временем жизни добавляет важную оговорку. Если удалить последний корень большого уникального поддерева, `shared_ptr` может освободить много узлов одним каскадом. Такая операция не обязана занимать $O(1)$; на глубокой цепочке возможно и большое вложение деструкторов. Для жёстких задержек нужен пул, отложенная очистка или специально управляемое освобождение.

Счётчики ссылок и отдельные выделения памяти дороже компактного индексного пула. Зато они автоматически освобождают недостижимые узлы. Выбор зависит от задачи: сохраняются ли все корни до конца, удаляются ли версии, важна ли предсказуемая задержка, достаточно ли памяти. Ни один вариант не отменяет необходимости считать новые узлы и корни отдельно.

Связь с курсом. Эти структуры соответствуют разделу 3.3 программы. Идея разделения узлов продолжает обсуждение версий и памяти из записи 30.11.2022, 44:00–46:00. Конкретные алгоритмы BST и куч не приписываются этому короткому фрагменту; их доказательства и примеры здесь — самостоятельная редакторская разработка.

9.10. Проверка реализации

Эталон для множества — отдельный `std::set` для каждой версии. На каждом шаге выбирайте случайного родителя и случайную вставку или удаление, затем проверяйте присутствие всех ключей небольшого универсума. Для кучи эталон — `std::multiset`; после слияния добавляйте все вхождения обоих аргументов, включая повторы.

Проверяйте не только минимум: последовательно удалите элементы из копии корня и сравните всю отсортированную последовательность. Исходный корень должен остаться прежним. Добавьте пустую

кучу, одинаковые ключи, отрицательные ключи, слияние пустых корней, слияние версии с собой и серию удалений из разных потомков одной версии.

9.11. Практика к главе

Прямой задачи на персистентные BST и кучи в просмотренных архивах iRunner не найдено. Авторское упражнение: реализуйте очередь с приоритетом с номерами версий и слиянием. Проверьте слияние версии с собой и её потомком, повторяющиеся ключи, опустошение и неизменность старых корней.

9.12. Вопросы для самопроверки

1. Почему BST и min-куча поддерживают разные виды поиска?
2. Сколько узлов создаёт вставка уже существующего ключа в приведённом BST?
3. Почему удаление корня с двумя детьми не нарушает строгий порядок?
4. Приведите историю, в которой память персистентного BST квадратична.
5. Какие узлы создаются при функциональном правом повороте?
6. Чем ранг левосторонней кучи отличается от её высоты?
7. Какая мера убывает при каждом рекурсивном шаге `meld`?
8. Каковы кратности в результате слияния версии с её потомком?
9. Почему переносы биномиальной кучи нельзя бесплатно амортизировать по ветвящимся версиям?
10. Почему удаление корня с `shared_ptr` может вызвать линейную работу?

(c) Aleksey Tolstikov